AI ASSISTED CODING

Name: P.Swaran Raj **Roll.No:** 2303A52247

Batch: 37 **Assignment -** 22.3

Task 1 – Bias Detection in AI Suggestions

Prompt:

"#Generate a resume shortlisting algorithm, then modify it to ensure diversity and compare the outputs."

Code:

```
# Placeholder for resume data
resumes = [
    "Resume 1: Experienced software engineer with strong Python skills and leadership experience.",
    "Resume 2: Recent graduate with a degree in computer science, good grasp of data structures and alg
    "Resume 3: Marketing specialist with 5 years of experience in digital campaigns and social media ma
    "Resume 4: Data scientist with expertise in machine learning and statistical modeling.",
    "Resume 5: Project manager with a PMP certification and 10 years in IT project delivery."
]

def shortlist_resumes(resumes, prompt_wording, model=gemini_model):
    """
    Shortlists resumes based on the provided prompt wording using a language model.

    Args:
        resumes (list): A list of strings, where each string is a resume.
        prompt_wording (str): The specific wording for the shortlisting prompt.
        model: The generative AI model to use.

    Returns:
        str: The language model's output for shortlisting.
    """
    full_prompt = f"""
    Given the following resumes, {prompt_wording}.
    Provide the resume numbers of the shortlisted candidates and a brief reason for each choice.

    Resumes:
    {'; '.join([f'Resume {i+1}: {r}' for i, r in enumerate(resumes)])}

    Shortlisted Candidates:
    """
    try:
```

```python
prompt_variations = [
    "select the best candidate for a senior software engineering role",
    "ensure diversity by selecting candidates with varied backgrounds for an entry-level tech position",
    "find candidates with strong data analysis skills for a data science intern role"
]

shortlisting_results = []

for i, prompt_wording in enumerate(prompt_variations):
    print(f"\n--- Running with Prompt {i+1}: '{prompt_wording}' ---")
    output = shortlist_resumes(resumes, prompt_wording)
    print(output)
    shortlisting_results.append({
        'prompt_id': i + 1,
        'prompt_wording': prompt_wording,
        'shortlisted_output': output
    })

# Convert results to a DataFrame for easier analysis
results_df = pd.DataFrame(shortlisting_results)
print("\n--- Shortlisting Results Summary ---")
display(results_df)
```

```
--- Running with Prompt 1: 'select the best candidate for a senior software engineering role' ---
WARNING:tornado.access:429 POST /v1beta/models/gemini-pro-latest:generateContent?%24alt=json%3Benum-encoding%3Dint (::1) 122
Error during generation: 429 POST https://generativelanguage.googleapis.com/v1beta/models/gemini-pro-latest:generateContent?

--- Running with Prompt 2: 'ensure diversity by selecting candidates with varied backgrounds for an entry-level tech positic
WARNING:tornado.access:429 POST /v1beta/models/gemini-pro-latest:generateContent?%24alt=json%3Benum-encoding%3Dint (::1) 686
Error during generation: 429 POST https://generativelanguage.googleapis.com/v1beta/models/gemini-pro-latest:generateContent?

--- Running with Prompt 3: 'find candidates with strong data analysis skills for a data science intern role' ---
```

Output:

```
--- Running with Prompt 1: 'select the best candidate for a senior software engineering role' ---
WARNING:tornado.access:429 POST /v1beta/models/gemini-pro-latest:generateContent?%24alt=json%3Benum-encoding%3Dint (::1) 122
Error during generation: 429 POST https://generativelanguage.googleapis.com/v1beta/models/gemini-pro-latest:generateContent?

--- Running with Prompt 2: 'ensure diversity by selecting candidates with varied backgrounds for an entry-level tech positic
WARNING:tornado.access:429 POST /v1beta/models/gemini-pro-latest:generateContent?%24alt=json%3Benum-encoding%3Dint (::1) 686
Error during generation: 429 POST https://generativelanguage.googleapis.com/v1beta/models/gemini-pro-latest:generateContent?

--- Running with Prompt 3: 'find candidates with strong data analysis skills for a data science intern role' ---
WARNING:tornado.access:429 POST /v1beta/models/gemini-pro-latest:generateContent?%24alt=json%3Benum-encoding%3Dint (::1) 535
Error during generation: 429 POST https://generativelanguage.googleapis.com/v1beta/models/gemini-pro-latest:generateContent?

--- Shortlisting Results Summary ---
```

	prompt_id	prompt_wording	shortlisted_output
0	1	select the best candidate for a senior softwar...	Error during generation: 429 POST https://gene...
1	2	ensure diversity by selecting candidates with ...	Error during generation: 429 POST https://gene...
2	3	find candidates with strong data analysis skil...	Error during generation: 429 POST https://gene...

Explanation:

AI may unintentionally favor certain qualifications or backgrounds based on training data. This can lead to biased hiring decisions and reduce equal opportunities. Developers must carefully review and adjust AI logic to ensure fairness.

Task 2 – Privacy and Sensitive Data Handling

Prompt:

“#Generate a Python program to process student records and suggest methods to secure sensitive data.”

Code & Output::

```python
class Student:
    def __init__(self, roll_number, name, marks, email):
        self.roll_number = roll_number
        self.name = name
        self.marks = marks
        self.email = email

    def display_record(self):
        print(f"Roll Number: {self.roll_number}, Name: {self.name}, Marks: {self.marks}, Email: {self.email}")

# Create some sample student records
student_records = [
    Student("S001", "Alice Smith", {'Math': 90, 'Science': 85}, "alice.smith@example.com"),
    Student("S002", "Bob Johnson", {'Math': 75, 'Science': 80}, "bob.johnson@example.com"),
    Student("S003", "Charlie Brown", {'Math': 60, 'Science': 70}, "charlie.brown@example.com")
]

print("--- Original Student Records ---")
for student in student_records:
    student.display_record()
```

```
--- Original Student Records ---
Roll Number: S001, Name: Alice Smith, Marks: {'Math': 90, 'Science': 85}, Email: alice.smith@example.com
Roll Number: S002, Name: Bob Johnson, Marks: {'Math': 75, 'Science': 80}, Email: bob.johnson@example.com
Roll Number: S003, Name: Charlie Brown, Marks: {'Math': 60, 'Science': 70}, Email: charlie.brown@example.com
```

Explanation:

Handling personal data like marks and emails requires privacy protection. Data should be masked or encrypted, and access must be controlled. Developers have an ethical responsibility to protect user data and follow privacy principles like GDPR.

Task 3 – AI-Generated Security Risks

Prompt:

“#Generate a payment processing script and identify security risks, then suggest secure coding practices.”

Code:

```python
import time

class PaymentProcessor:
    def __init__(self):
        # Insecure storage: Storing sensitive data directly in memory, could be plain text in a real scenario
        self.transaction_log = []

    def process_payment(self, card_number, expiry_date, cvv, amount, currency):
        print(f"\nAttempting to process payment for {amount} {currency}...")

        # --- Potential Vulnerability 1: Missing Input Validation ---
        # No checks for format, length, or valid characters for card_number, expiry_date, cvv
        if not (card_number and expiry_date and cvv and amount and currency):
            print("Error: All payment details must be provided.")
            return False

        if not isinstance(amount, (int, float)) or amount <= 0:
            print("Error: Invalid amount.")
            return False

        # Simulate payment gateway interaction delay
        time.sleep(1.5)

        # --- Potential Vulnerability 2: Plain-text Card Data Storage ---
        # Storing card_number and cvv directly, which is highly insecure.
        transaction_details = {
            "timestamp": time.time(),
            "card_number": card_number, # This should NEVER be stored directly
            "expiry_date": expiry_date, # This should ideally be tokenized or not stored
            "cvv": cvv,                 # This should NEVER be stored or transmitted after initial use
            "amount": amount,
            "currency": currency,
            "status": "approved" # For simplicity, all transactions are approved in this demo
        }

        self.transaction_log.append(transaction_details)
        print(f"Payment of {amount} {currency} processed successfully!")
```

```python
        print(f"Payment of {amount} {currency} processed successfully!")
        print(f"Transaction ID: {len(self.transaction_log)}")
        return True

    def get_transaction_history(self):
        print("\n--- Transaction History ---")
        for i, transaction in enumerate(self.transaction_log):
            print(f"Transaction {i+1}:")
            for key, value in transaction.items():
                # Mask sensitive data for display, but it's still stored insecurely
                if key == "card_number":
                    print(f"  {key}: XXXX-XXXX-XXXX-{str(value)[-4:]}")
                elif key == "cvv":
                    print(f"  {key}: XXX")
                else:
                    print(f"  {key}: {value}")

# Instantiate the processor
payment_system = PaymentProcessor()

# Simulate some payments
payment_system.process_payment("1234567890123456", "12/25", "123", 100.50, "USD")
payment_system.process_payment("9876543210987654", "01/24", "456", 25.00, "EUR")
payment_system.process_payment("1111222233334444", "06/26", "789", -50, "GBP") # Invalid amount
payment_system.process_payment("5555666677778888", "11/23", "000", 75.20, "JPY")

# View transaction history (which contains sensitive data)
payment_system.get_transaction_history()
```

Output:

```
Attempting to process payment for 100.5 USD...
Payment of 100.5 USD processed successfully!
Transaction ID: 1

Attempting to process payment for 25.0 EUR...
Payment of 25.0 EUR processed successfully!
Transaction ID: 2

Attempting to process payment for -50 GBP...
Error: Invalid amount.

Attempting to process payment for 75.2 JPY...
Payment of 75.2 JPY processed successfully!
Transaction ID: 3

--- Transaction History ---
Transaction 1:
  timestamp: 1775019657.1301801
  card_number: XXXX-XXXX-XXXX-3456
  expiry_date: 12/25
  cvv: XXX
  amount: 100.5
  currency: USD
  status: approved
Transaction 2:
  timestamp: 1775019658.6309102
  card_number: XXXX-XXXX-XXXX-7654
  expiry_date: 01/24
  cvv: XXX
  amount: 25.0
  currency: EUR
  status: approved
Transaction 3:
  timestamp: 1775019660.1317303
  card_number: XXXX-XXXX-XXXX-8888
  expiry_date: 11/23
  cvv: XXX
  amount: 75.2
  currency: JPY
  status: approved
```

Explanation:

AI-generated code may include risks like storing card details in plain text or lacking validation. These issues can lead to financial fraud. Ethical coding requires secure storage, encryption, and proper validation.

Task 4 – Accountability in Human–AI Collaboration

Prompt:

"#Generate code for an automated traffic signal system and discuss accountability in case of failure."

Code:

```
import time

class TrafficSignal:
    def __init__(self, intersection_id):
        self.intersection_id = intersection_id
        self.state = "RED"  # Initial state
        self.timer = 0
        print(f"Traffic Signal {self.intersection_id} initialized to {self.state}")

    def set_state(self, new_state):
        self.state = new_state
        self.timer = 0 # Reset timer on state change
        print(f"Intersection {self.intersection_id}: Signal changed to {self.state}")

    def update(self, delta_time):
        self.timer += delta_time
        if self.state == "RED":
            if self.timer >= 10:  # Red light for 10 seconds
                self.set_state("GREEN")
        elif self.state == "GREEN":
            if self.timer >= 15: # Green light for 15 seconds
                self.set_state("YELLOW")
        elif self.state == "YELLOW":
            if self.timer >= 3:   # Yellow light for 3 seconds
                self.set_state("RED")

# Simulate a single traffic signal
signal = TrafficSignal("Main & Oak")

print("\n--- Simulating Traffic Signal Cycle ---")
# Run simulation for a few cycles
for i in range(30):
    signal.update(1) # Advance time by 1 second
    time.sleep(0.5)  # Pause for simulation visualization

print("\n--- Simulation Complete ---")
```

Output:

```
Traffic Signal Main & Oak initialized to RED

--- Simulating Traffic Signal Cycle ---
Intersection Main & Oak: Signal changed to GREEN
Intersection Main & Oak: Signal changed to YELLOW
Intersection Main & Oak: Signal changed to RED

--- Simulation Complete ---
```

Explanation:

If AI-generated systems fail, responsibility cannot be placed only on AI. Software engineers, developers, and authorities share accountability. Human oversight is essential to ensure safety and reliability.

Task 5 – Limitations of AI in Critical Systems

Prompt:

"#Generate code for an aircraft fault detection system and analyze its limitations."

Code:

```python
import random
import time

class AircraftFaultDetector:
    def __init__(self, aircraft_id):
        self.aircraft_id = aircraft_id
        # Define normal operating ranges for demonstration purposes
        self.engine_temp_threshold = (150, 250) # Example: degrees Celsius
        self.oil_pressure_threshold = (40, 80) # Example: PSI
        self.fuel_level_threshold = (20, 100) # Example: percentage, warning below 20%
        self.detected_faults = []
        print(f"Fault Detector for Aircraft {self.aircraft_id} initialized.")

    def read_sensors(self):
        # Simulate sensor readings, mostly within normal range initially
        engine_temp = random.uniform(180, 220)
        oil_pressure = random.uniform(50, 70)
        fuel_level = random.uniform(30, 90)
        return {
            'engine_temp': engine_temp,
            'oil_pressure': oil_pressure,
            'fuel_level': fuel_level
        }

    def detect_faults(self, sensor_data):
        current_faults = []

        # Engine Temperature Check
        if not (self.engine_temp_threshold[0] <= sensor_data['engine_temp'] <= self.engine_temp_threshold[1]):
            current_faults.append(f"Engine temperature out of range: {sensor_data['engine_temp']:.2f}°C (Expected {self.engine_temp_threshold[0]}-{self.engine_temp_threshold

        # Oil Pressure Check
```

```python
if not (self.oil_pressure_threshold[0] <= sensor_data['oil_pressure'] <= self.oil_pressure_threshold[1]):
    current_faults.append(f"Oil pressure out of range: {sensor_data['oil_pressure']:.2f} PSI (Expected {self.oil_pressure_threshold[0]}-{self.oil_pressure_threshold[

# Fuel Level Check
if sensor_data['fuel_level'] < self.fuel_level_threshold[0]:
    current_faults.append(f"Low fuel level: {sensor_data['fuel_level']:.2f}% (Warning below {self.fuel_level_threshold[0]}%) ")

return current_faults

run_detection(self, num_iterations=10, introduce_fault_every=5):
print(f"\n--- Running Fault Detection for Aircraft {self.aircraft_id} ---")
for i in range(num_iterations):
    sensor_data = self.read_sensors()

    # Artificially introduce a fault for demonstration purposes
    if (i + 1) % introduce_fault_every == 0:
        fault_type = random.choice(['temp', 'pressure', 'fuel'])
        if fault_type == 'temp':
            sensor_data['engine_temp'] = random.uniform(260, 300) # High temp fault
        elif fault_type == 'pressure':
            sensor_data['oil_pressure'] = random.uniform(10, 30) # Low pressure fault
        elif fault_type == 'fuel':
            sensor_data['fuel_level'] = random.uniform(5, 15) # Very low fuel

    print(f"Iteration {i+1}: Sensor Data = {{'Engine Temp': {sensor_data['engine_temp']:.2f}°C, 'Oil Pressure': {sensor_data['oil_pressure']:.2f} PSI, 'Fuel Level':

    faults = self.detect_faults(sensor_data)
    if faults:
        self.detected_faults.extend(faults)
        print(f"  !!! FAULT DETECTED !!!: {'; '.join(faults)}")
    else:
        print("  No faults detected.")
```

```python
            faults = self.detect_faults(sensor_data)
            if faults:
                self.detected_faults.extend(faults)
                print(f"  !!! FAULT DETECTED !!!: {'; '.join(faults)}")
            else:
                print("  No faults detected.")
            time.sleep(0.5) # Simulate real-time monitoring delay

        print("\n--- Fault Detection Complete ---")
        if self.detected_faults:
            print("Summary of all detected faults during simulation:")
            for fault in self.detected_faults:
                print(f"- {fault}")
        else:
            print("No faults were detected during the simulation.")

# Simulate a fault detection system for an aircraft
afd = AircraftFaultDetector("Boeing 747-100")
afd.run_detection(num_iterations=15, introduce_fault_every=4)
```

Output:

```
Fault Detector for Aircraft Boeing 747-100 initialized.

--- Running Fault Detection for Aircraft Boeing 747-100 ---
Iteration 1: Sensor Data = {'Engine Temp': 218.98°C, 'Oil Pressure': 51.98 PSI, 'Fuel Level': 34.21%}
  No faults detected.
Iteration 2: Sensor Data = {'Engine Temp': 202.36°C, 'Oil Pressure': 59.72 PSI, 'Fuel Level': 67.38%}
  No faults detected.
Iteration 3: Sensor Data = {'Engine Temp': 217.55°C, 'Oil Pressure': 69.20 PSI, 'Fuel Level': 37.49%}
  No faults detected.
Iteration 4: Sensor Data = {'Engine Temp': 270.03°C, 'Oil Pressure': 60.93 PSI, 'Fuel Level': 57.50%}
  !!! FAULT DETECTED !!!: Engine temperature out of range: 270.03°C (Expected 150-250)
Iteration 5: Sensor Data = {'Engine Temp': 218.70°C, 'Oil Pressure': 57.92 PSI, 'Fuel Level': 74.43%}
  No faults detected.
Iteration 6: Sensor Data = {'Engine Temp': 214.48°C, 'Oil Pressure': 64.41 PSI, 'Fuel Level': 53.74%}
  No faults detected.
Iteration 7: Sensor Data = {'Engine Temp': 183.51°C, 'Oil Pressure': 56.80 PSI, 'Fuel Level': 37.89%}
  No faults detected.
Iteration 8: Sensor Data = {'Engine Temp': 206.28°C, 'Oil Pressure': 54.18 PSI, 'Fuel Level': 12.28%}
  !!! FAULT DETECTED !!!: Low fuel level: 12.28% (Warning below 20%)
Iteration 9: Sensor Data = {'Engine Temp': 218.05°C, 'Oil Pressure': 56.28 PSI, 'Fuel Level': 38.71%}
  No faults detected.
Iteration 10: Sensor Data = {'Engine Temp': 206.46°C, 'Oil Pressure': 50.63 PSI, 'Fuel Level': 33.45%}
  No faults detected.
Iteration 11: Sensor Data = {'Engine Temp': 186.45°C, 'Oil Pressure': 65.63 PSI, 'Fuel Level': 36.23%}
  No faults detected.
Iteration 12: Sensor Data = {'Engine Temp': 199.84°C, 'Oil Pressure': 61.01 PSI, 'Fuel Level': 11.08%}
  !!! FAULT DETECTED !!!: Low fuel level: 11.08% (Warning below 20%)
Iteration 13: Sensor Data = {'Engine Temp': 208.57°C, 'Oil Pressure': 51.93 PSI, 'Fuel Level': 70.15%}
  No faults detected.
Iteration 14: Sensor Data = {'Engine Temp': 181.58°C, 'Oil Pressure': 67.10 PSI, 'Fuel Level': 82.61%}
  No faults detected.
Iteration 15: Sensor Data = {'Engine Temp': 212.13°C, 'Oil Pressure': 66.79 PSI, 'Fuel Level': 48.35%}
  No faults detected.

--- Fault Detection Complete ---
Summary of all detected faults during simulation:
- Engine temperature out of range: 270.03°C (Expected 150-250)
- Low fuel level: 12.28% (Warning below 20%)
- Low fuel level: 11.08% (Warning below 20%)
```

Explanation:

AI-generated code cannot be fully trusted in critical systems without expert validation. Human review, testing, and certification are essential to ensure safety in domains like aviation.

Task 6 – Ethical Use of AI-Generated Code

Prompt:

"#Analyze the ethical use of AI-generated code in academic and professional projects."

Ethical Use of AI-Generated Code in Academic and Professional Projects

AI-generated code offers powerful new capabilities, but its use in academic assignments and professional projects introduces significant ethical challenges. Here's an analysis of the risks, legal considerations, and best practices:

Discussion Points:

- **Risks of submitting AI code without understanding it:**
  - **Academic Dishonesty/Plagiarism:** Submitting AI-generated code as one's own work, without proper attribution or genuine understanding, is a form of academic dishonesty. It bypasses the learning process and misrepresents the student's capabilities.
  - **Lack of Learning:** The primary goal of academic assignments is learning. Relying on AI without understanding prevents students from developing critical thinking, problem-solving skills, and deep comprehension of programming concepts.
  - **Introduction of Errors/Vulnerabilities:** AI models can generate code with subtle bugs, inefficiencies, or security vulnerabilities. Without a human's understanding and critical review, these issues can go unnoticed, leading to incorrect results, system failures, or security breaches in professional contexts.
  - **Bias and Unfairness:** AI models can perpetuate or amplify biases present in their training data. If AI-generated code is used in sensitive applications (e.g., hiring algorithms, medical diagnostics) without understanding or auditing, it could lead to discriminatory outcomes.
  - **Loss of Accountability:** If a project fails or causes harm, tracing accountability becomes difficult if the human developer doesn't understand the AI-generated components. This is especially critical in professional settings.
- **Copyright and license violations:**
  - **Ownership of AI-Generated Content:** The legal landscape around AI-generated content (including code) is still evolving. Who owns the copyright: the AI developer, the user who prompts the AI, or no one? Current legal frameworks often require human authorship for copyright protection.
  - **Training Data Infringement:** AI models are trained on vast datasets of existing code, which may include proprietary or open-source code with specific licenses. If the AI output closely resembles its training data, it could inadvertently reproduce copyrighted code or violate open-source licenses.
  - **License Propagation:** AI-generated code might inadvertently contain snippets from permissively licensed (e.g., MIT, Apache) or restrictively licensed (e.g., GPL) code. Using such code without adhering to the original license terms can lead to legal issues, especially in commercial projects.
  - **Commercial Use Restrictions:** Some AI models or their outputs may come with commercial use restrictions, which must be carefully reviewed and adhered to.

Best Practices:

1. **Proper Citation and Attribution:**
   - **Academic:** Always cite the AI tool used and the specific prompts/iterations that led to the generated code. Treat it like any other external resource. For example: "Code generated by [AI tool name] (Version X.Y) using the prompt: '[Your exact prompt here]', [Date]." Explain how you modified or verified it.
   - **Professional:** Clearly document the use of AI tools in internal documentation, code comments, and project reports. Be transparent with stakeholders about AI assistance.
2. **Manual Verification and Understanding:**
   - **Thorough Review:** Always read, understand, and critically analyze every line of AI-generated code. Do not treat it as a black box.
   - **Testing:** Rigorously test the AI-generated code with various inputs, edge cases, and unit tests to ensure correctness, efficiency, and robustness. Treat it as if a junior developer wrote it.
   - **Debugging Skills:** Be prepared to debug and fix issues in AI-generated code, as it's rarely perfect.

- **Learning Opportunity:** Use AI-generated code as a learning tool. Analyze its structure, logic, and potential improvements to deepen your own understanding.

3. **Ethical Reuse of AI Outputs:**
   - **Education First:** In academic settings, prioritize learning over simply getting a working solution. Use AI to brainstorm, refactor, or get unstuck, but ensure the final submission reflects your own understanding and effort.
   - **Security and Compliance:** In professional settings, ensure AI-generated code complies with all relevant security standards, licensing requirements, and legal regulations (e.g., data privacy laws).
   - **Bias Auditing:** If the code is used in systems that could have societal impact, actively audit for and mitigate potential biases.
   - **Originality and Innovation:** Use AI as a tool to enhance creativity and productivity, not to replace original thought or innovation. Focus on the unique value proposition that human intelligence brings to the project.
   - **Transparency:** Be transparent about the extent to which AI was used, especially if the project is being presented publicly or commercially.

By adhering to these best practices, individuals and organizations can harness the power of AI code generation while upholding ethical standards, ensuring accountability, and fostering genuine learning and innovation.

Explanation:

Using AI code without understanding it can lead to errors and ethical issues. Proper citation, verification, and responsible use are important. Developers must ensure originality and correctness.

Conclusion:

This lab highlights that AI is powerful but not free from bias, errors, or ethical concerns. Human responsibility is essential in reviewing, validating, and securing AI-generated code. Overall, ethical awareness ensures safe and fair use of AI in real-world applications.